✨ 情動イメージング演習 進捗報告プレゼンテーション

OpenFaceを用いた顔画像からの情動情報解析・時系列実装

～ テキスト第1回～第3回課題の実装完了と独自発展システムの開発報告 ～

📍 情報・データサイエンス学部 3年

🗓️ 2026年10月22日 (Day 3)

成果物：Pythonコード9点・可視化グラフ8点・同期プレイヤー

PROGRESS SUMMARY

演習全体の進捗状況サマリー

シラバス全4回に対する我々の実作業（Day 1・Day 2）での前倒し達成対比

01. 第1回相当 (Day 1実施)

顔を「データ」として測る

- **OpenFace環境の稼働確認**と動画からの特徴量抽出
- **頭部姿勢(Head Pose)解析:** Pitch, Yaw, Rollの3軸角度変化を抽出
- **成果物:** `plot_sayu.py` / `Sayu_pose.png`
- **知見:** 身体動作の動画目視から客観的数値時系列への変換を実証

02. 第2回相当 (Day 1実施)

表情を数値として測る

- **Action Unit (AU) の時系列可視化:** 笑顔(AU06, AU12)と困惑(AU04)
- **驚き(Surprise)の複合AU分析:** AU01, 02, 05, 25, 26の同期
- **独自情動指標の作成:** Happy, Surprise, Sadness を数式モデル化
- **成果物:** `2-2.py`, `2-3.py`, `2-kadai.py`

03. 第3回相当 (Day 2実施)

時間変化から情動を読む + 独自発展

- **時系列差分(diff) & イベント検出:** 表情急変点の抽出とフラグ化
- **ノイズ平滑化:** 移動平均 (rolling mean) によるトレンド抽出
- **Emotion Timeline構築:** 4段統合プロット (`3-hukusen.py`)
- **独自発展:** OpenCV×Matplotlib動画同期プレイヤー (`3-WB.py`)

頭部姿勢の時系列可視化 (Head Pose)

動画中の頭部運動（左右・上下・傾き）を客観的な角度（rad）データとして抽出

実施内容と技術ポイント

- **対象データ:** Sayu. csv (頭部運動動画のOpenFace解析結果)
- **抽出変数:** pose_Rx (Pitch: 上下), pose_Ry (Yaw: 左右), pose_Rz (Roll: 傾き)
- **解析意義:** 「首を振った」という主観的観察を、ラジアン単位の客観時系列ピークとして分離・同定可能に。

PLOT_SAYU.PY （抜粋） PYTHON

```
df = pd.read_csv('Sayu.csv') df.columns = df.columns.str.strip() # pose_Rx, pose_Ry, pose_Rz の時系列プロット plt.plot(df['timestamp'], df['pose_Rx'], label='Pitch (Rx)') plt.plot(df['timestamp'], df['pose_Ry'], label='Yaw (Ry)') plt.plot(df['timestamp'], df['pose_Rz'], label='Roll (Rz)') plt.legend(); plt.savefig('Sayu_pose.png')
```

- **考察:** 左右運動時に pose_Ry が明瞭に±0.5 rad以上の振れ幅を記録。

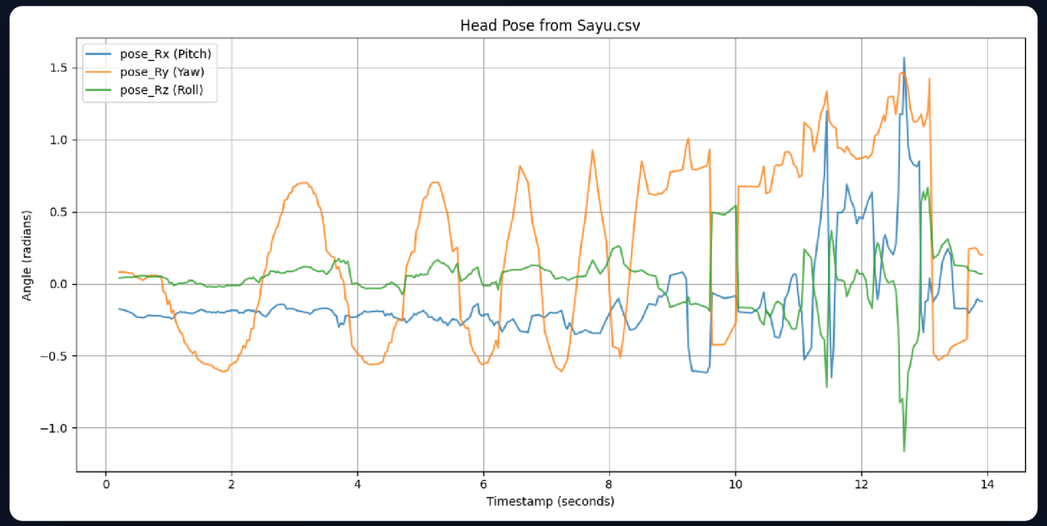


図: Sayu_pose.png （頭部3軸回転角の時系列プロット）

Action Unit (AU) による顔表情の数値化

FACS（顔面動作符号化システム）に基づく微小筋肉運動の強度定量化

観測対象と筋運動の対応

- **AU06_r (頬上げ):** 眼輪筋の収縮。自然な笑顔で強く生起。
- **AU12_r (口角上げ):** 大頬骨筋の収縮。笑顔の主要決定因子。
- **AU04_r (眉下げ):** 皺眉筋の収縮。困惑・集中・不快時に上昇。

2-2.PY PYTHON

```
df = pd.read_csv('2-2.csv') df.columns = df.columns.str.strip()
plt.plot(df['timestamp'], df['AU06_r'], label='AU06 (Cheek Raiser)')
plt.plot(df['timestamp'], df['AU12_r'], label='AU12 (Lip Corner Puller)')
plt.plot(df['timestamp'], df['AU04_r'], label='AU04 (Brow Lowerer)') plt.legend();
plt.xlabel('Time[s]'); plt.ylabel('AU Intensity') plt.show()
```

- **考察:** 中盤の笑顔区間において、AU06とAU12が同期してピークを形成。主観に頼らず客観的強度変化として記録。

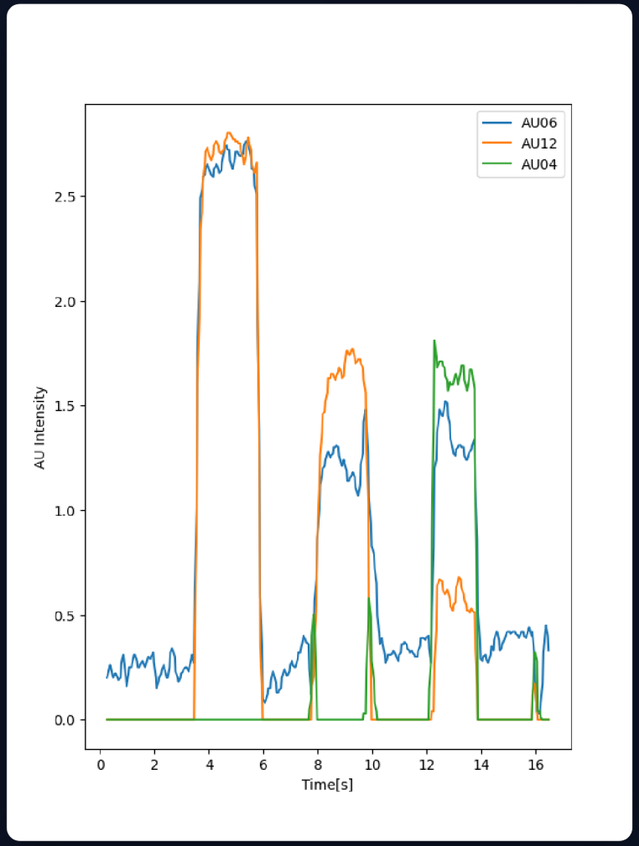


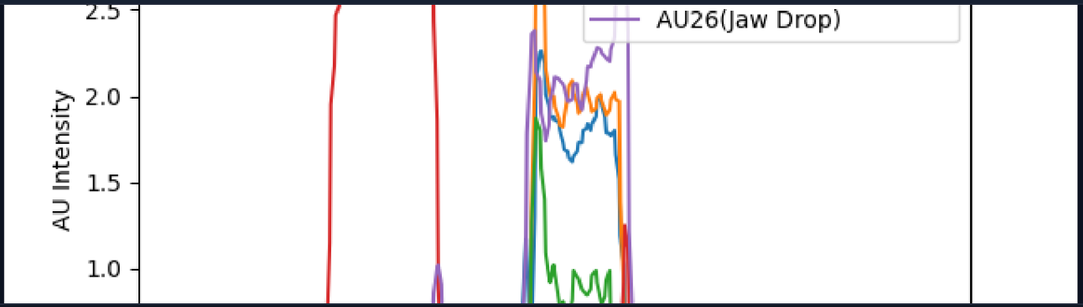
図: 2-2.png（AU06, AU12, AU04 の時系列変化）

複合AU解析と独自情動指標のモデリング

驚き(Surprise)の複数AU連動解析と、独自感情スコア（Happy, Sadness等）の設計

驚き区間のAU多変量解析 (2-3.py)

内側/外側眉上げ(AU01,02)、上まぶた上げ(AU05)、開口(AU25,26)の共起を検証。



独自情動指標の設計 (2-kadai.py)

2-KADAI.PY PYTHON

```
df["happy"] = df["AU25_r"] df["surprise"] = df["AU01_r"] + df["AU02_r"] +
df["AU05_r"] + df["AU26_r"] df["sadness"] = df["AU04_r"]
```

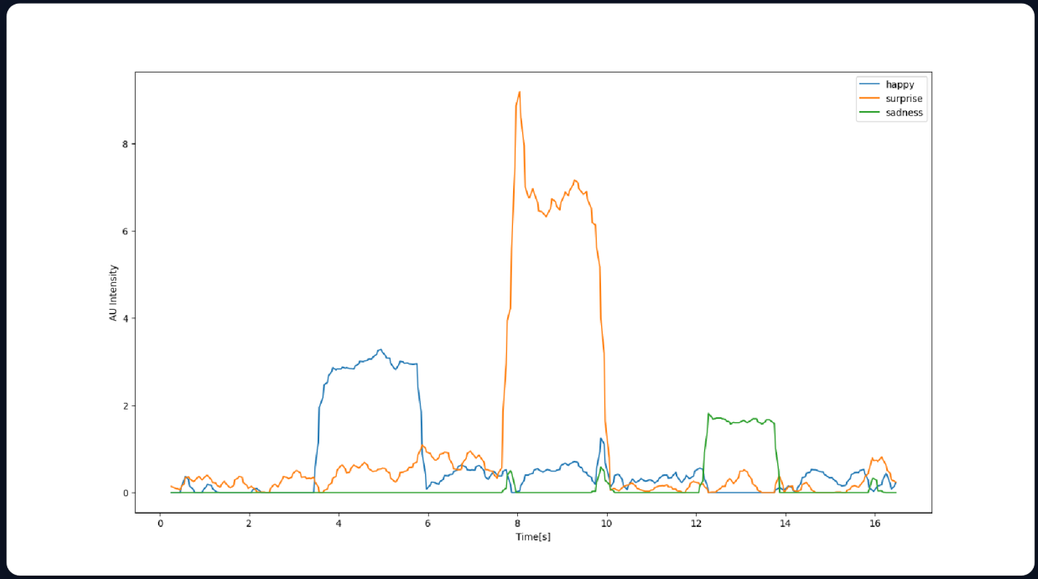


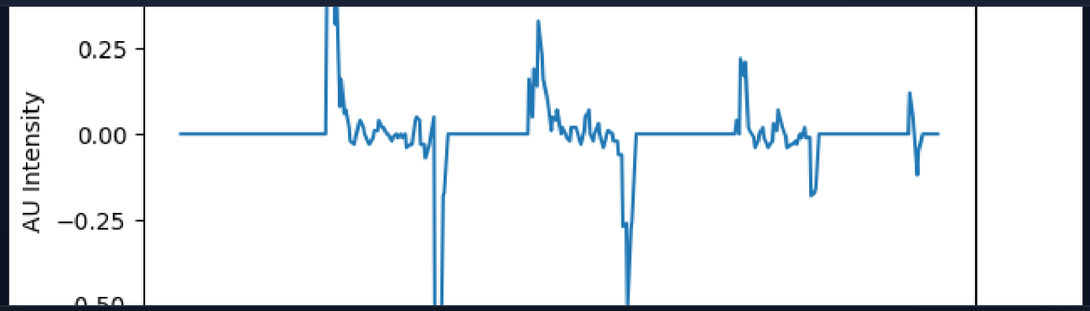
図: 2-kadai.png（独自指標 Happy / Surprise / Sadness の時系列）

時系列差分とイベント検出フラグ化

静止画的特徴から動画時系列へ：急峻な変化点の抽出としきい値による離散イベント化

01. フレーム間差分による変化点検出 (3-1.py)

「値の高さ」だけでなく、表情が動き出した**「立ち上がり（モーメント）」**を df["AU12_r"].diff() で抽出。



02. 閾値判定によるイベントフラグ化 (3-2.py)

3-2.PY（抜粋） PYTHON

```
# 閾値2を超えた区間を笑顔イベント(1)として離散系列化 df["smileFlag"] = (df["AU12_r"] > 2).astype(int)
```

連続値を0/1のイベント生起系列として扱うことで、開始・終了時刻や継続時間を自動計算可能に。

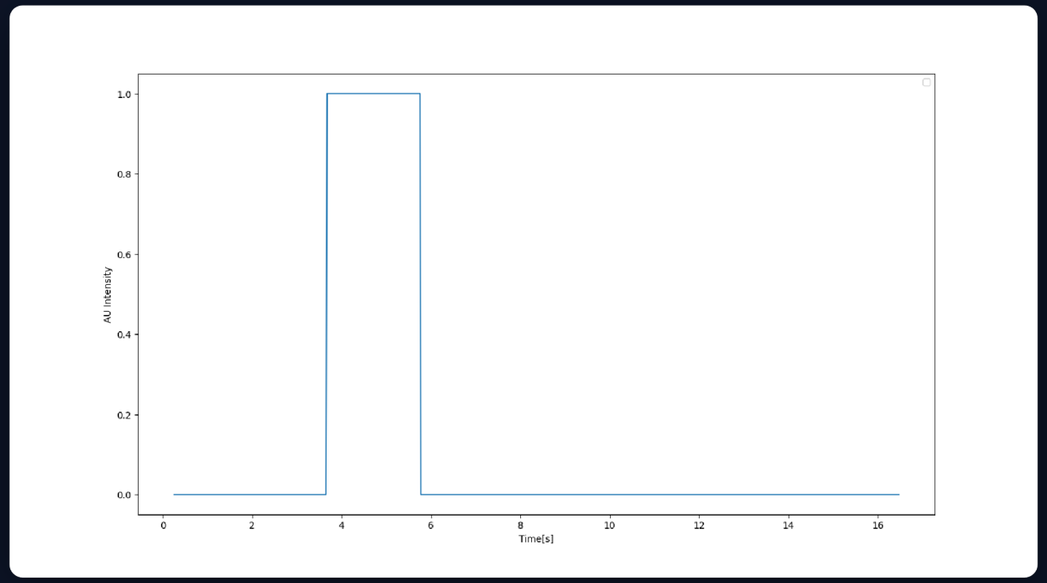


図: 3-2.png（smileFlag による笑顔イベントの区間検出）

移動平均フィルタによるノイズ低減と平滑化

トラッキングの微小振動を除去し、本質的な情動ダイナミクスを抽出

課題と平滑化アルゴリズム

- **課題:** 顔トラッキングでは、照明や微小な頭部揺らぎによってフレーム単位のスパイクノイズが混入する。
- **手法:** 移動平均（Rolling Mean with window=8, center=True）を適用。

3-2_MA.PY PYTHON

```
df["AU_smooth"] = ( df["AU12_r"] .rolling(window=8, center=True) .mean() )
plt.plot(df['timestamp'], df["AU12_r"], label="Raw") plt.plot(df['timestamp'],
df["AU_smooth"], label="AU_smooth") plt.legend(); plt.show()
```

- **考察（トレードオフの検証）:** window=8（約0.25秒）を採用することで、笑顔の立ち上がり遅延を最小限に抑えつつ高周波振動を効果的に抑制。

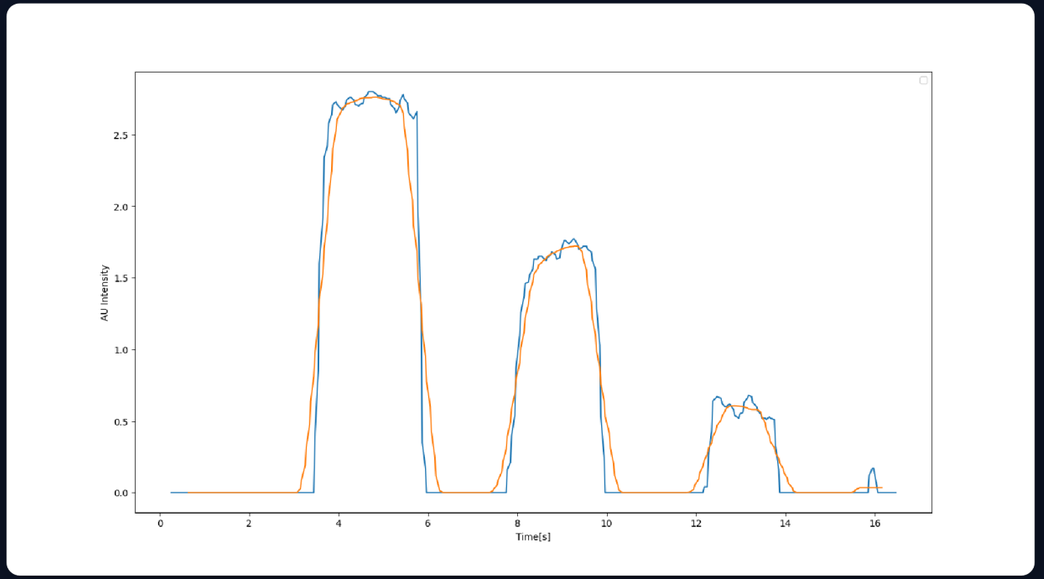


図: 3-2_MA.png（生データ vs 移動平均平滑化データの比較）

複合時系列可視化（Emotion Timeline）の構築

テキスト第3コマの到達目標：複数指標（表情・頭部・まばたき）の多段同期プロット

4段マルチプロットの構成定義

- **段1: Smile Score**（AU12_r > 2）→ 笑顔生起イベント
- **段2: AU04**（AU04_r > 1）→ 眉間の緊張・集中・困惑
- **段3: Head movement**（pose_Rz.diff() < -0.005）→ 首の急峻な傾き
- **段4: Blink**（AU45_c > 0）→ まばたき検出パルス

3-HUKUSEN.PY（抜粋） PYTHON

```
fig, axes = plt.subplots(4, 1, figsize=(10, 8), sharex=True)
axes[0].plot(df['timestamp'], df["Smile Score"], color="tab:blue")
axes[1].plot(df['timestamp'], df["AU04"], color="tab:orange")
axes[2].plot(df['timestamp'], df["Head movement"], color="tab:green")
axes[3].plot(df['timestamp'], df["Blink"], color="tab:red") plt.tight_layout();
plt.show()
```

- **成果:** 異種指標が同じ時間軸で並ぶことで、「首の傾きと同時に笑顔が生じた」といった**共起パターン**を直観的に把握可能に。

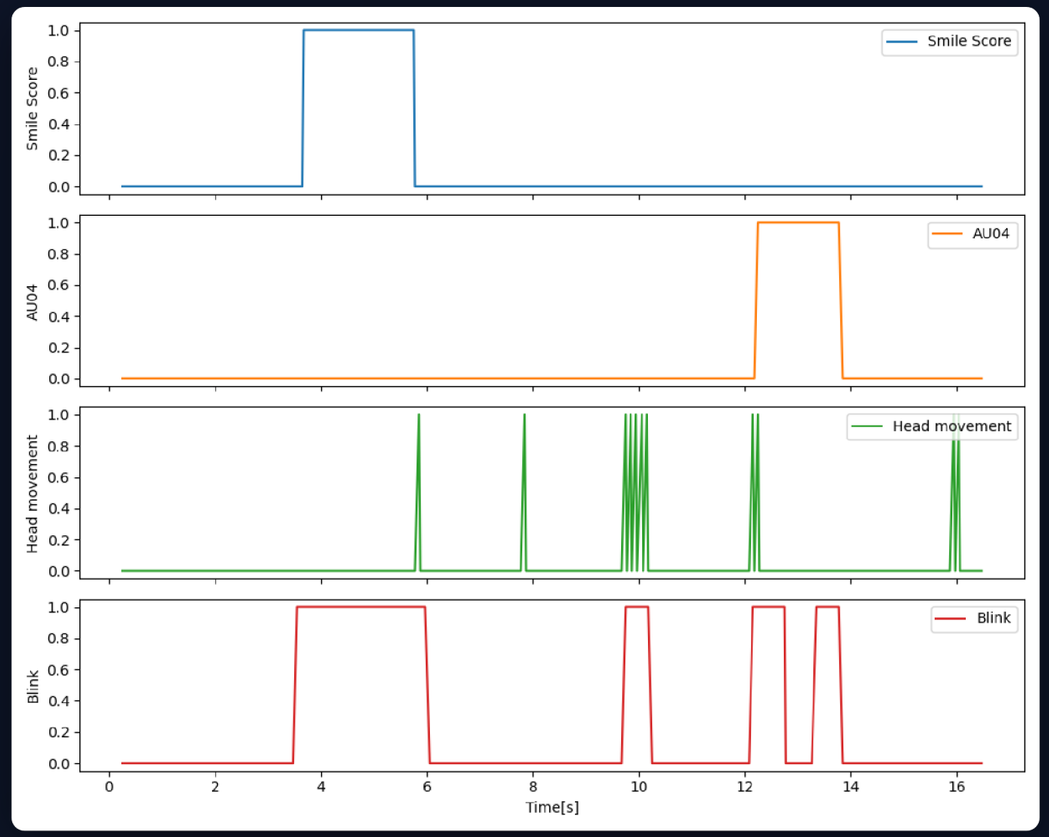


図: 3-hukusen.png（4段マルチプロットによるEmotion Timeline）

ADVANCED WORK

動画・時系列グラフ リアルタイム同期プレイヤー

テキストの構想をさらに発展：OpenCVとMatplotlibのアニメーションによる完全同期可視化

独自開発のアーキテクチャ (3-WB.py)

- **開発背景:** テキスト記載の「上段に動画、中段にAU、下段に頭部運動」という理想構成を静止画ではなく**再生可能な同期アニメーション**として具現化。
- **技術スタック:** cv2.VideoCapture + matplotlib.animation.FuncAnimation
- **連動機構:** 現在フレームの描画と同時に、下部グラフの赤色破線バー（axvline）が時系列上を追従。

3-WB.PY（抜粋） PYTHON

```
gs = fig.add_gridspec(3, 1, height_ratios=[2, 1, 1]) def update(frame_idx): ret, frame = cap.read() im.set_data(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)) current_time = df['timestamp'].iloc[frame_idx] line_au.set_xdata([current_time, current_time]) line_head.set_xdata([current_time, current_time]) return im, line_au, line_head
```

- **絶大な効果:** 映像の実際の表情と数値ピークの遅延やズレを瞬時に目視検証できるアノテーション基盤が完成。

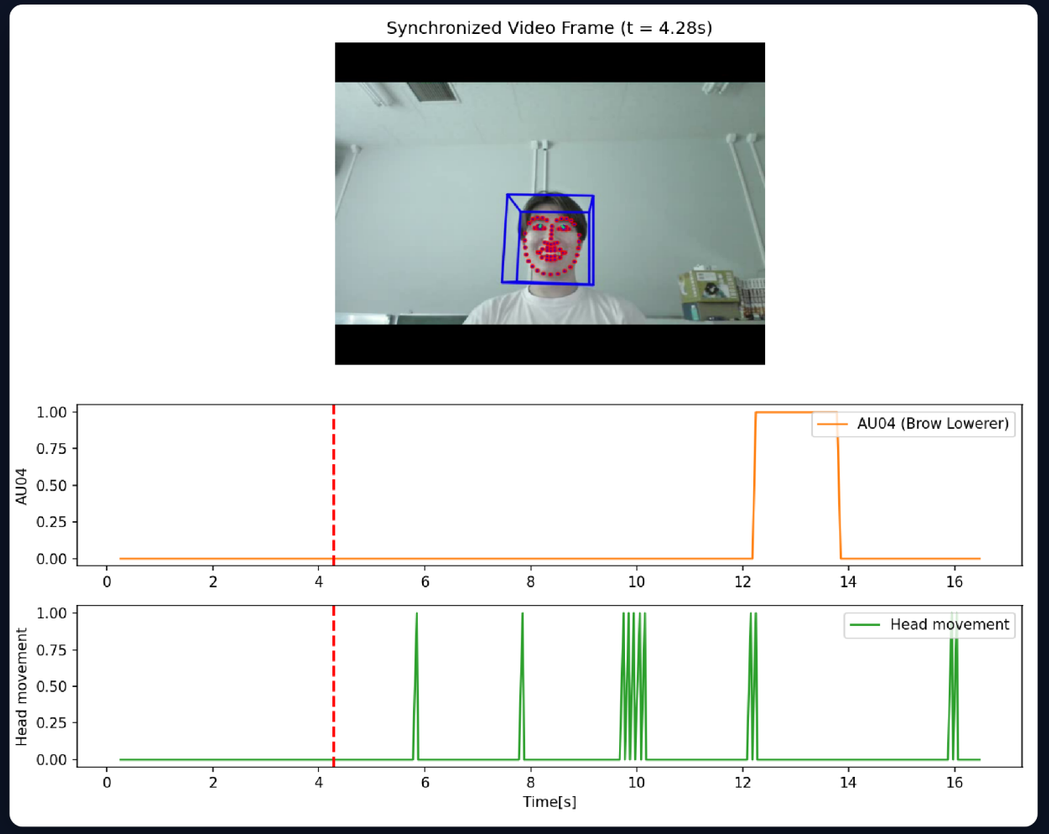


図: 3-WB.png（上段: 動画フレーム / 中段: AU04 / 下段: Head movement 同期スナップショット）

CONCLUSION & NEXT STEP

成果のまとめと第4回（ミニプロジェクト）への展望

第4回に向けた挑戦：マルチモーダル特徴量による『嘘発見器』の精度向上とアプリ開発

■ ここまでの到達点（基盤確立）

- **実活動2日間でテキスト全3回分を完全走破:**
  OpenFaceによる特徴量抽出（頭部姿勢・AU・視線・瞬目）から、FACS/AU解析、時系列差分、ノイズ平滑化、Emotion Timelineまで一貫した解析パイプラインを確立。
- **独自同期アニメーションシステムの完成 (3-WB.py):**
  動画と時系列データを完全同期させるビューアを独自開発。
- **データサイエンスとしての重要な視座:**
  「Pixel → Feature → Behavior → State → Emotion」の推論階層を学び、**観測（筋運動）と解釈（心理・情動推定）を混同しない**厳密な分析姿勢を習得。

■ 第4回 計画：『嘘発見器』の高度化＆アプリ開発

- **① 嘘発見（虚偽検出）の精度向上アプローチ:**
  - **多変量特徴の統合:** まばたき急増(AU45)、視線の泳ぎ(gaze)、作り笑顔(AU12有/AU06無)、眉間の緊張(AU04)の複合解析
  - **微表情の差分検出:** 質問直後の急峻な表情変化（マイクロエクスプレッション）を `diff()` で捕捉
  - **機械学習モデルの導入:** scikit-learnによる「真実回答時 vs 虚偽回答時」の分類器（SVM / Random Forest）学習と精度評価
- **② 実用的な『嘘発見アプリ』の開発:**
  - `3-WB.py` の同期技術を発展させ、Webカメラから**「嘘らしさスコア(Deception Index)」**をリアルタイム判定・描画するGUIアプリを実装
  - 質問提示と連動した自動判定・可視化ダッシュボードの構築